



Arquitectura de Computadoras

TP1 – ALU

Painenao, Juan Manuel

Stangaferro, Alejandro

9 de septiembre de 2026

1. Introducción

Una de las piezas claves dentro de una CPU es la ALU (Unidad Aritmético-Lógica). Se encarga de realizar operaciones aritméticas, como suma y resta, y operaciones lógicas como AND y OR. Recibe los datos necesarios desde los registros, procesa las instrucciones y devuelve el resultado para ser almacenado o utilizado en otras tareas. El objetivo de este trabajo práctico es implementar una ALU en una FPGA. La ALU debe tener un bus de datos parametrizable y se debe validar el desarrollo por medio de *Test Bench*.

2. Marco Teórico

2.1. ALU

La unidad aritmético-lógica es un componente fundamental del procesador encargado de ejecutar operaciones matemáticas y lógicas sobre datos binarios. Dentro de la estructura de la CPU, la ALU cumple una función ejecutiva al procesar datos conforme a las instrucciones emitidas por la unidad de control. La ALU que debe implementarse en la FPGA consta de ocho operaciones, las cuales se listan a continuación:

ADD

Código: 100000

Funcionamiento: suma binaria entre A y B.

Ejemplo: $00000011\ (3) + 00000101\ (5) = 00001000\ (8)$

SUB

Código: 100010

Funcionamiento: resta binaria entre A y B ($A - B$).

Ejemplo: $00001000\ (8) - 00000011\ (3) = 00000101\ (5)$

AND

Código: 100100

Funcionamiento: conjunción lógica bit a bit entre A y B.

Ejemplo: $11001010 \wedge 10110110 = 10000010$

OR

Código: 100101

Funcionamiento: disyunción lógica bit a bit entre A y B.

Ejemplo: $11001010 \vee 10110110 = 11111110$

XOR

Código: 100110

Funcionamiento: disyunción exclusiva bit a bit entre A y B (1 solo si los bits difieren).

Ejemplo: $11001010 \oplus 10110110 = 01111100$

SRA

Código: 000011

Funcionamiento: desplazamiento aritmético de A hacia la derecha, B posiciones; replica el bit de signo (MSB) para conservar el signo.

Ejemplo: $10000000 \gg 1 = 11000000$

SRL

Código: 000010

Funcionamiento: desplazamiento lógico de A hacia la derecha, B posiciones; rellena con ceros por la izquierda.

Ejemplo: $10000000 \ggg 1 = 01000000$

NOR

Código: 100111

Funcionamiento: negación de la disyunción bit a bit entre A y B ($\sim (A \vee B)$).

Ejemplo: $\sim (11001010 \vee 10110110) = 00000001$

3. Diseño

El diseño del trabajo práctico se pensó en dos archivos:

3.1. ALU.v

En `ALU.v` se realiza la lógica de la ALU. Primero se declaran los parámetros como la cantidad de bits de datos, que son ocho y la cantidad de bits del código de operación que son seis. Luego se declaran los puertos de entrada y salida. Como entradas se declaran A y B de ocho bits cada y OP de seis bits. Como salida se declara como registro RES, donde se almacena el resultado de la operación. Como parámetros locales se asocian el código de cada operación a un nombre de modo que sea mas fácil la comprensión cuando se lee código. Finalmente en un bloque combinacional `always@(*)` se realiza un case evaluando el valor de OP. Si OP coincide con alguno de los códigos de operaciones, se realiza la operación correspondiente con A y B y se almacena en RES. En caso de que OP no tenga un valor que corresponda a una de las operaciones, en RES se coloca como resultado todos ceros.

3.2. top_ALU.v

El archivo `top_ALU.v` es el puente entre la lógica y la placa. El archivo (`ALU.v`) utiliza toda lógica combinacional por lo que es necesario adaptarlo a los requerimientos de la placa. También esta el problema de multiplexado de las entradas, ya que A y B son de ocho bits cada uno y OP es de 6 bits, se necesitaría un switch de 22 bits para poder asociar un valor a cada entrada. Sin embargo la Basys 3 solo tiene 16 switches. Por lo que es necesario realizarlo de otra manera. Lo que aclaraba la consigna era utilizar botones de enable para cada entrada. De esta forma se utiliza solamente ocho bits como máximo del switch, y dependiendo del enable que se presione, el valor del switch se asigna a una entrada u otra. Para eso se declaran tres registros `reg_a`, `reg_b` y `reg_op` que se encargan de almacenar el valor asignado a cada entrada, mientras se asignan los valores de las demás entradas.

La instancia `mi_alu` conecta los tres registros a la ALU y su salida RES maneja `reg_res`, que a su vez va a los LEDs.

Por ultimo, se utilizan tres bloques `always`, uno para cada registro pero todos comparten el mismo clock. Por ejemplo, en el primer caso si el `en_a` (Corresponde al enable de la entrada A) esta en alto se realiza una asignación no bloqueante del switch al registro A (`reg_A`)

4. Verificación

Se realizó un *Test Bench*. autoverificante, es decir el test realiza las operaciones y se autoverifica. Para esto el test bench obtiene dos resultados, uno es el devuelto por el DUT, el Device Under Test, que en nuestro caso es la ALU, y otro calculado por un modelo de referencia. De modo de poder evaluar el resultado de la ALU contra el resultado correcto. Para ello se asignan valores random a las entradas A,B y OP y se realizan las operaciones y comparaciones mil veces. Después de cada estímulo, el testbench espera #10 para que la lógica combinacional se estabilice, calcula el esperado con el modelo y compara contra RES. La comparacion utiliza `!=` (Igualdad case) para diferenciarlo de los valores x (Indeterminado) y z (Alta impedancia) así se marca como error. Hay un contador de errores que aumenta cada vez que un resultado no coincide con el resultado del modelo de referencia.

4.1. Resultados de la verificación

En la Fig. 1 se observa que la verificación simuló y testeó 1000 vectores y finalizó sin errores.

```
launch_simulation: Time (s): cpu = 00:00:10 ; elapsed = 00:00:13 . Memory (MB): peak = 1708.480 ; gain = 0.000
run -all
TEST OK: 1000 vectores sin errores.
Sfinish called at time : 10 us : File "C:/Users/JuanM/ALU/ALU.srcs/sim_1/new/tb_alu.v" Line 112
```

Figura 1: Simulación behavioral de la ALU mostrando los 1000 vectores.

4.2. Analisis de tiempo

En la Fig. 2 se observa el análisis de tiempo de todos los vectores testeados. En el primer caso, por ejemplo $A = 10011111$ y $B = 01011100$ la operación es $OP = 100000$ lo que equivale a la operación ADD, el resultado $RES = 11011111$ y el esperado $Esp = 11011111$ por lo que la comparación es correcta. Cabe destacar un detalle de visualización: el valor **esperado** aparece desfasado una posición respecto de **RES**. Esto no se debe a un error de la ALU, sino a que **esperado** se calcula recién después del retardo de estabilización (#10) del combinacional; al muestrear la señal, el testbench ya presenta el estímulo del vector siguiente, lo que produce el corrimiento aparente.

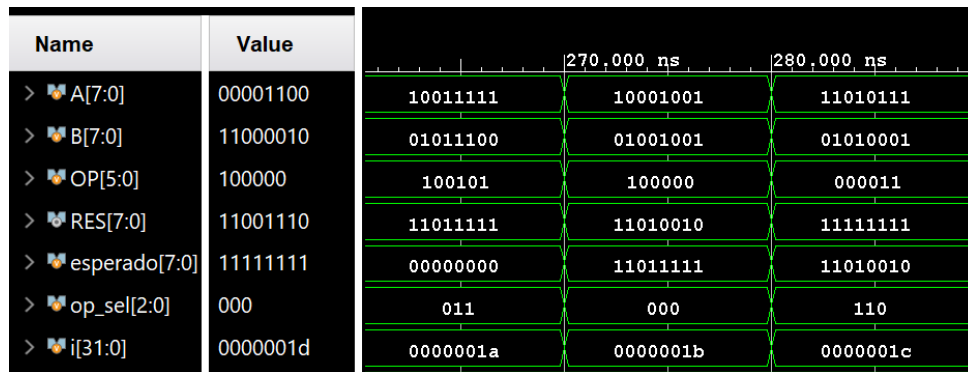


Figura 2: Analisis de tiempo con 3 ejemplos.

5. Implementación física

Para la implementación física se utiliza una placa de entrenamiento FPGA Artix 7 Basys 3, la cual al ser académica o de entrenamiento, ya cuenta con los switches, los leds y los pulsadores integrados en la placa. Por lo que solo se utilizó esa placa y el cable correspondiente para conectarla a la computadora (USB-MicroUSB). Para cargar el programa en la FPGA primero la síntesis traduce el RTL a una netlist de compuertas usando recursos reales del Artix-7 (LUTs para la lógica del case, flip-flops para reg_a, reg_b, reg_op). Luego la implementación hace place & route: decide en qué celda física del die va cada LUT y flip-flop, y traza los caminos que las conectan entre sí y con los pines de I/O, es acá donde se verifica que el diseño cumpla timing. El archivo de constraints (XDC) relaciona cada puerto lógico a un pin físico de la placa (switch[0] → SW0, reg_res[0] → LD0) y declara la frecuencia del reloj.

Con el diseño ya ubicado y ruteado, Vivado genera el bitstream (.bit), el binario que describe el estado de cada celda de configuración del FPGA. Finalmente, desde el Hardware Manager se conecta la placa por USB, Vivado detecta el Artix-7 por la cadena JTAG y se carga ese .bit. Como la programación por JTAG es volátil, el diseño vive en la RAM de configuración y se borra al cortar la alimentación, así que hay que recargarlo cada vez que reconectás la placa.

6. Conclusiones

En conclusión fue un trabajo practico con la dificultad justa para comenzar a programar por primera vez en verilog, y recordar como era toda la parte de conexión y programación de la FPGA. A lo largo de todo el desarrollo del trabajo practico no tuvimos mayores complicaciones, primero en clase realizamos el código y el top, luego realizamos los testbench y por ultimo implementamos físicamente en la placa. Un detalle que confundió un poco cuando veíamos los testbench y las líneas de tiempo, fue cuando queríamos comparar el resultado con el golden model. Siempre veíamos el resultado y abajo donde debería verse el resultado del modelo de referencia, se veía el resultado de la operación anterior. Es decir, estaba como desfasada esa parte. Averiguando nos dimos cuenta que era por el retardo colocado en los testbench de #10 para que se acomode el combinacional. Se cargaba el resultado mientras A/B/OP/RES ya habian cambiado a un nuevo valor y esto hacia que se viera desfasaje contra el resultado de la ALU. Sacando eso no hubieron mas complicaciones en el trabajo practico y el mismo resultado muy útil como primer acercamiento a la programación en verilog.

7. Anexo

Listing 1: Módulo ALU.v

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/26/2026 08:51:24 PM
7  // Design Name:
8  // Module Name: Alu
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module Alu #(
24     parameter NB_DATA = 8,
25     parameter NB_OP_CODE = 6
26 )
27 (
28     input [NB_DATA-1:0] A,
29     input [NB_DATA-1:0] B,
30     input [NB_OP_CODE-1:0] OP,
31     output reg [NB_DATA-1:0] RES,
32     output [0:0] CARRY,
33     output [0:0] OVF
34 );
35 localparam OP_ADD = 6'b100000;
36 localparam OP_SUB = 6'b100010;
```

```

37     localparam OP_AND = 6'b100100;
38     localparam OP_OR = 6'b100101;
39     localparam OP_XOR = 6'b100110;
40     localparam OP_SRA = 6'b000011;
41     localparam OP_SRL = 6'b000010;
42     localparam OP_NOR = 6'b100111;
43
44     always @(*) begin
45         case (OP)
46             OP_ADD: RES = A+B;
47             OP_SUB: RES = A-B;
48             OP_AND: RES = A&B;
49             OP_OR: RES = A|B;
50             OP_XOR: RES = A^B;
51             OP_SRL: RES = A>>B;
52             OP_SRA: RES = $signed(A) >>> B;
53             OP_NOR: RES = ~(A|B);
54             default: RES = 8'b0;
55         endcase
56     end
57
58 endmodule

```

Listing 2: Módulo top_ALU.v

```

1  'timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/26/2026 09:37:21 PM
7  // Design Name:
8  // Module Name: top_ALU
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module top_ALU #(
24     parameter NB_DATA = 8
25 )
26 (
27     input [NB_DATA-1: 0] switch,
28     input [0:0] clock,
29     input en_a,
30     input en_b,
31     input en_op,
32     output [NB_DATA-1:0] reg_res
33 );
34

```

```

35     reg [NB_DATA-1:0] reg_a;
36     reg [NB_DATA-1:0] reg_b;
37     reg [6-1:0] reg_op;
38
39
40     Alu mi_alu(
41         .A (reg_a),
42         .B (reg_b),
43         .OP (reg_op),
44         .RES (reg_res)
45     );
46
47     always @(posedge clock) begin
48         if (en_a)
49             reg_a <= switch;
50     end
51
52     always @(posedge clock) begin
53         if (en_b)
54             reg_b <= switch;
55     end
56
57     always @(posedge clock) begin
58         if (en_op)
59             reg_op <= switch;
60     end
61
62
63 endmodule

```

Listing 3: Módulo tb_alu.v

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Testbench autoverificante para la ALU
4  // - Genera entradas aleatorias (A, B, OP)
5  // - Calcula el resultado esperado con un modelo de referencia (golden
6  //   model)
7  // - Compara automaticamente y lleva la cuenta de errores
8  //////////////////////////////////////
9  module Alu_tb;
10
11     // Parametros del DUT (deben coincidir con los de la ALU)
12     localparam NB_DATA = 8;
13     localparam NB_OP    = 6;
14
15     // Cantidad de vectores aleatorios a probar
16     localparam N_VECTORES = 1000;
17
18     // Codigos de operacion (iguales a los de la ALU)
19     localparam OP_ADD = 6'b100000;
20     localparam OP_SUB = 6'b100010;
21     localparam OP_AND = 6'b100100;
22     localparam OP_OR  = 6'b100101;
23     localparam OP_XOR = 6'b100110;
24     localparam OP_SRA = 6'b000011;
25     localparam OP_SRL = 6'b000010;
26     localparam OP_NOR = 6'b100111;

```

```

27 // Senales hacia/desde el DUT
28 reg [NB_DATA-1:0] A;
29 reg [NB_DATA-1:0] B;
30 reg [NB_OP-1:0] OP;
31 wire [NB_DATA-1:0] RES;
32
33 // Auxiliares del testbench
34 reg [NB_DATA-1:0] esperado;
35 reg [2:0] op_sel;
36 integer i;
37 integer errores;
38
39 // Instancia del modulo bajo prueba (DUT = Device Under Test)
40 Alu #(
41     .NB_DATA (NB_DATA),
42     .NB_OP_CODE (NB_OP)
43 ) dut (
44     .A (A),
45     .B (B),
46     .OP (OP),
47     .RES (RES),
48     .CARRY (), // sin conectar por ahora
49     .OVF () // (se agregan cuando implementes carry/overflow)
50 );
51
52 // Modelo de referencia (golden model): calcula el resultado
53 // ESPERADO
54 // de forma independiente al DUT. Si el DUT tiene un bug, los dos
55 // difieren.
56 function [NB_DATA-1:0] modelo;
57 input [NB_DATA-1:0] a;
58 input [NB_DATA-1:0] b;
59 input [NB_OP-1:0] op;
60 begin
61     case (op)
62         OP_ADD: modelo = a + b;
63         OP_SUB: modelo = a - b;
64         OP_AND: modelo = a & b;
65         OP_OR : modelo = a | b;
66         OP_XOR: modelo = a ^ b;
67         OP_SRL: modelo = a >> b;
68         OP_SRA: modelo = $signed(a) >>> b;
69         OP_NOR: modelo = ~(a | b);
70         default: modelo = {NB_DATA{1'b0}};
71     endcase
72 end
73 endfunction
74
75 initial begin
76     errores = 0;
77
78     for (i = 0; i < N_VECTORES; i = i + 1) begin
79         // 1) Generar entradas aleatorias
80         A = $random; // se trunca a NB_DATA bits
81         B = $random;
82         op_sel = $random; // uso los 3 bits bajos: 0..7
83
84         // 2) Elegir una operacion VALIDA a partir de op_sel

```

```

83         case (op_sel)
84             3'd0: OP = OP_ADD;
85             3'd1: OP = OP_SUB;
86             3'd2: OP = OP_AND;
87             3'd3: OP = OP_OR;
88             3'd4: OP = OP_XOR;
89             3'd5: OP = OP_SRL;
90             3'd6: OP = OP_SRA;
91             3'd7: OP = OP_NOR;
92         endcase
93
94         // 3) Esperar a que la logica combinacional se estabilice
95         #10;
96
97         // 4) Calcular el esperado y comparar (chequeo automatico)
98         esperado = modelo(A, B, OP);
99         if (RES != esperado) begin
100             $display("ERROR [t=%0t] A=%b B=%b OP=%b | RES=%b
101                 esperado=%b",
102                 $time, A, B, OP, RES, esperado);
103             errores = errores + 1;
104         end
105     end
106
107     // 5) Reporte final
108     if (errores == 0)
109         $display("TEST OK: %0d vectores sin errores.", N_VECTORES);
110     else
111         $display("TEST FALLIDO: %0d errores sobre %0d vectores.",
112                 errores, N_VECTORES);
113     $finish;
114 end
115 endmodule

```

Referencias

- [1] Patterson, D. A. y Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition)*.
- [2] Harris, S. L. y Harris, D. M. *Digital Design and Computer Architecture (RISC-V Edition)*.
- [3] Digilent. *Basys 3 FPGA Board Reference Manual*.